Embedded Systems and IoT — ISI LT — a.a. 2025/2026

Relazione dell’Assignment 2

Smart Drone Hangar

Brunelli — matricola 0001021672

Sistema embedded composto da due sottosistemi comunicanti via linea seriale: il Drone Hangar su Arduino UNO (architettura a task e FSM sincrone) e la Drone Remote Unit, GUI web su PC.

1. Introduzione

Il progetto implementa un sistema embedded per la gestione automatizzata di un hangar per droni. È composto da due sottosistemi che comunicano tramite linea seriale:

1. Drone Hangar (Arduino UNO) — gestisce sensori, attuatori e la logica di controllo locale, realizzata con architettura a task e macchine a stati finiti sincrone.

2. Drone Remote Unit (PC) — interfaccia grafica web (Node.js + Express + socket.io) per inviare i comandi del drone e monitorare la telemetria in tempo reale.

All'avvio la porta è chiusa con il drone dentro a riposo (L1 acceso, L2/L3 spenti, LCD **DRONE INSIDE**). Il decollo e l'atterraggio sono comandati dalla DRU; durante queste fasi L2 lampeggia con periodo 0,5 s. Un sistema di monitoraggio della temperatura può portare il sistema in pre-allarme o in allarme.

2. Architettura del sistema (Drone Hangar)

2.1 Hardware

Il sistema è basato su Arduino UNO e comprende:

Sensori: Sonar HC-SR04 (DDD, distanza del drone), PIR (DPD, presenza del drone nell'area di atterraggio), sensore di temperatura AM2302/DHT22.
Attuatori e output: servomotore (porta HD), LCD I2C 16×2 (stato), LED L1 (drone dentro, verde), L2 (operazioni in corso, verde), L3 (allarme, rosso).
Input: pulsante RESET per ripristinare il sistema dallo stato di allarme.

Componente	Pin Arduino UNO
PIR (DPD)	D2
Sonar (DDD) trig / echo	D3 / D4
L1 / L2 / L3	D5 / D6 / D7
RESET (pulsante)	D8 (INPUT_PULLUP)
Servo porta (HD)	D9
Temperatura (TEMP)	A0
LCD I2C 16×2	SDA=A4, SCL=A5 (0x27)

Nota: la traccia prevede un sensore di temperatura analogico (es. LM35/TMP36). In questo prototipo è stato impiegato un AM2302/DHT22 digitale, funzionalmente equivalente ai fini del monitoraggio e dell'allarme.

2.2 Software: architettura task-based

Il firmware segue un'architettura a task gestita da uno **scheduler cooperativo** (TaskScheduler) con tick base di 10 ms. Ogni task riceve tramite costruttore solo i riferimenti che gli servono, secondo il principio di minima conoscenza.

Task	Periodo	Ruolo
SensorTask	100 ms	acquisisce sonar, PIR e temperatura
TemperatureTask	100 ms	FSM temperatura NORMAL/PREALARM/ALARM
ControlTask	50 ms	FSM principale dell'hangar (porta, LED, LCD, comandi)
BlinkTask	250 ms	lampeggio di L2 (periodo 0,5 s) in decollo/atterraggio

SerialTask	—	comandi in ingresso e telemetria in uscita

3. Macchine a stati finiti sincrone (FSM)

Il comportamento è modellato con due FSM sincrone cooperanti: la FSM principale dell'hangar (ControlTask) e la FSM di monitoraggio della temperatura (TemperatureTask), che condiziona la prima.

3.1 FSM principale — Hangar

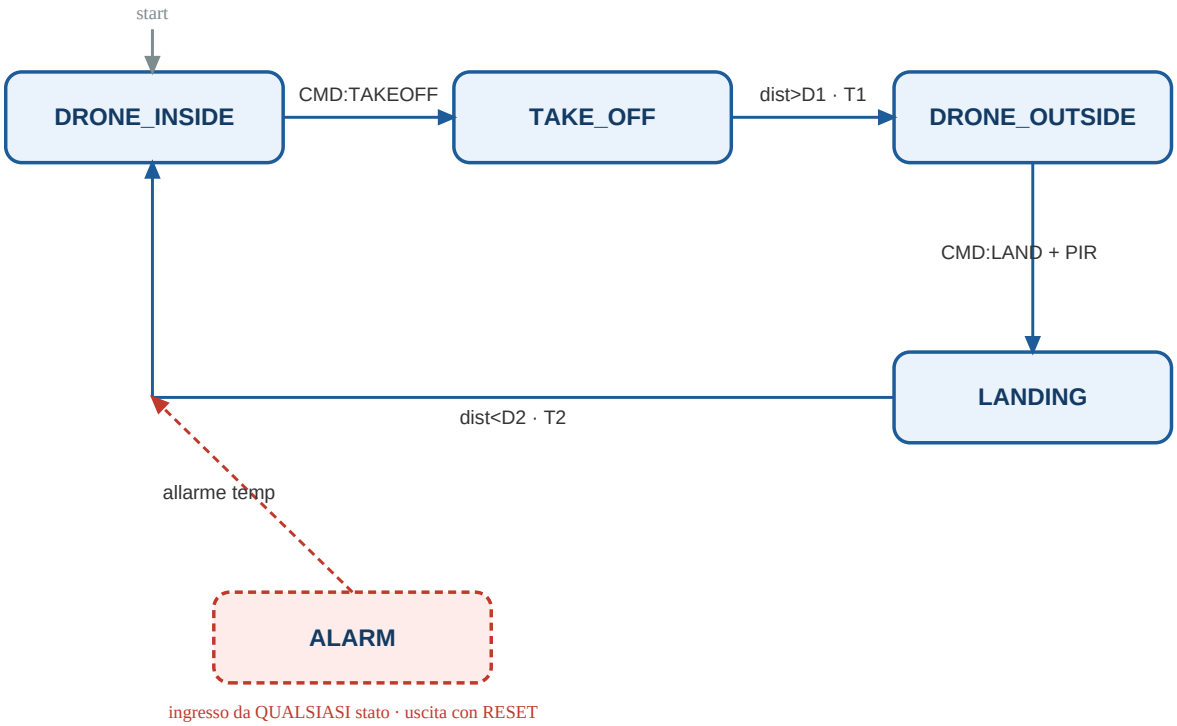


Figura 1 — FSM dell'hangar (ControlTask).

Stato	Porta	LED	LCD
DRONE_INSIDE	chiusa	L1 on	DRONE INSIDE
TAKE_OFF	aperta	L2 lampeggia	TAKE OFF
DRONE_OUTSIDE	chiusa	—	DRONE OUT
LANDING	aperta	L2 lampeggia	LANDING
ALARM	chiusa	L3 on	ALARM

Il **pre-allarme** non è uno stato dell'hangar ma una condizione del TemperatureTask: tramite `operationsAllowed()` blocca le **nuove** transizioni verso TAKE_OFF e LANDING, lasciando però completare un'operazione già in corso. L'allarme, invece, ha priorità: da qualunque stato porta in ALARM, da cui si esce solo con il pulsante RESET.

3.2 FSM temperatura

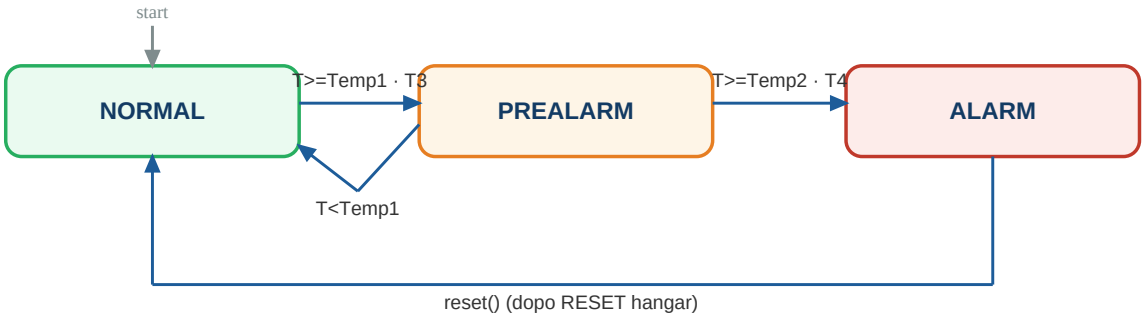


Figura 2 — FSM di monitoraggio della temperatura (TemperatureTask).

4. Drone Remote Unit (sottosistema PC)

La DRU è realizzata come applicazione **web** con server **Node.js + Express** e comunicazione in tempo reale via **socket.io** verso il browser; l'accesso alla porta seriale usa la libreria `serialport`. La GUI (http://localhost:3001) permette di:

• inviare i comandi di **decollo** e **atterraggio** (simulando il drone);
• visualizzare lo **stato del drone** (a riposo, in decollo, operativo, in atterraggio) e lo **stato dell'hangar** (normale, pre-allarme, allarme);
• visualizzare la **distanza dal suolo** durante l'atterraggio, la temperatura e lo stato del PIR.

Il server fa da ponte: interpreta le righe di telemetria dall'Arduino e le inoltra al browser come eventi socket.io; viceversa traduce i click dell'operatore in comandi seriali.

5. Protocollo di comunicazione (9600 baud)

Lo scambio dati avviene con messaggi testuali su seriale.

PC → Arduino:

`CMD:TAKEOFF` — richiesta di decollo.
`CMD:LAND` — richiesta di atterraggio.

Arduino → PC (una riga ogni 500 ms):

`STATE:<s>,TEMP:<t>,DIST:<d>,PREALARM:<0|1>,ALARM:<0|1>,PIR:<0|1>`

con $s \in$ {DRONE_INSIDE, TAKE_OFF, DRONE_OUTSIDE, LANDING, ALARM}. Il sonar restituisce 999.0 in assenza di eco (timeout). Il campo PIR viene mantenuto alto per un breve intervallo dopo un rilevamento, così un impulso breve resta visibile nella telemetria (più lenta della FSM).

5.1 Parametri di prototipazione

Parametro	Valore	Significato
D1 / T1	50 cm / 3 s	drone considerato fuori (in decollo)
D2 / T2	10 cm / 3 s	drone considerato atterrato
Temp1 / T3	30 °C / 3 s	soglia di pre-allarme
Temp2 / T4	40 °C / 2 s	soglia di allarme

Valori liberamente modificabili in config.h (utili per i test). Vincolo di coerenza fisica: distanza a riposo < D2 < D1.

6. Schema del circuito

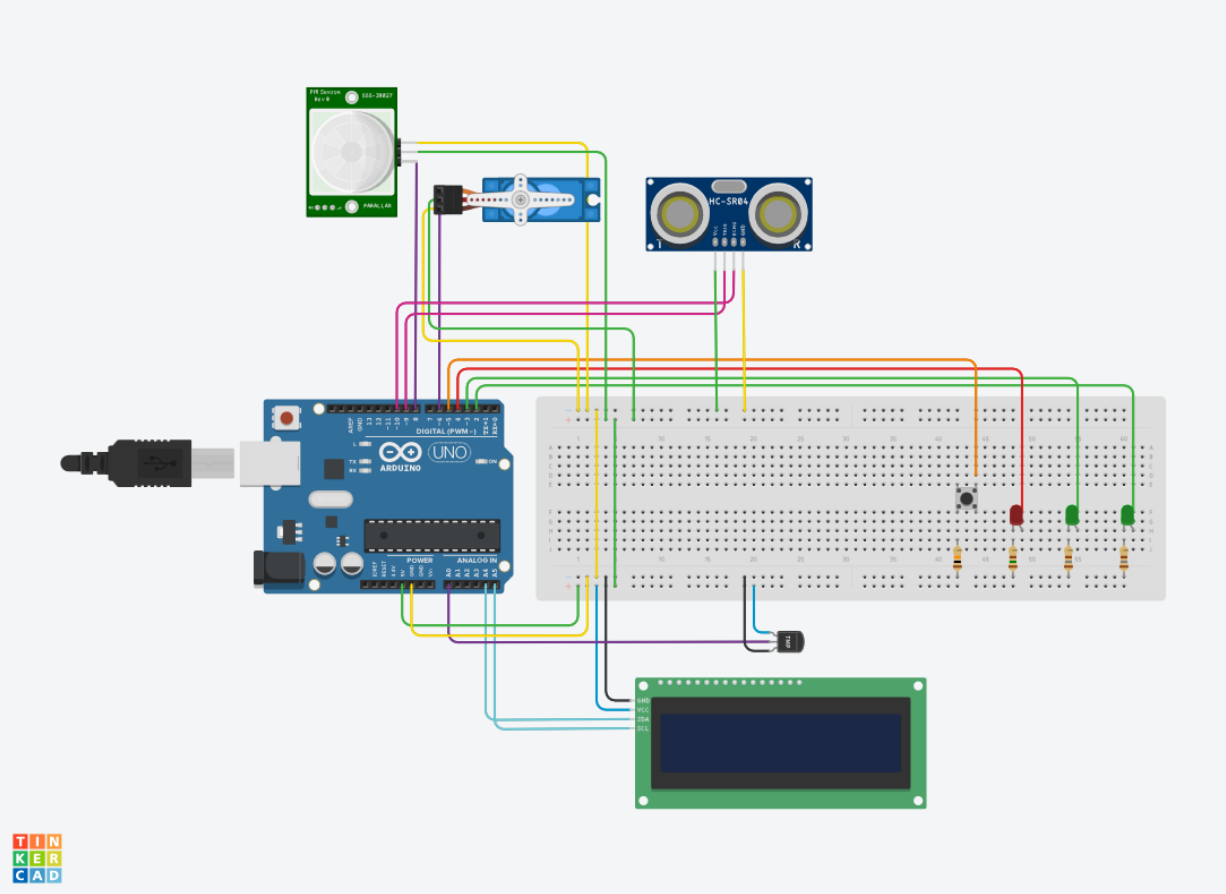


Figura 3 — Schema / breadboard del sistema.